COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010-DATA STRUCTURE AND ALGORITHM	**SECTION:**	CPE21S2
DATE PERFORMED:	09/04/26	**DATE SUBMITTED:**	09/08/26
NAME:	Bungag, Marc Jhowell M.	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA 7.1 Sorting Algorithms

1. PROCEDURE OUTPUT

What procedures did you perform?

TABLE 7.1

Code + Console Screenshot

main.cpp > ...

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArray(int arr[]);

int main(){

    int unsortedArr[maxSize] = {5, 2, 9, 1, 5};
    srand(time(0));
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()% 100;
    }

    std::cout<<"Unsorted Array: \n"<<std::endl;
    displayArray(unsortedArr);
}

void displayArray(int arr[]) {

    for(int i = 0; i < maxSize; i++){
        std::cout<<arr[i]<<" ";
    }
    std::cout<<std::endl;
}
```

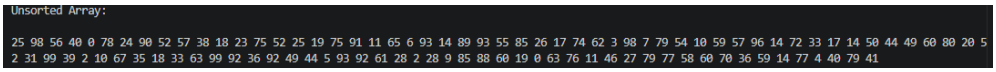
	
Observations	The code generates and displays 100 random numbers, with each number ranging from 0 to 99. The best-case and worst-case time complexity are both O(n) because the program needs to go through all the elements in the array regardless of the values generated. Since the array only contains 100 elements, the program runs quickly and does not require much processing time.

TABLE 7.2

Code + Console Screenshot	main.cpp > displayArray(int [])

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArray(int arr[]);

int main(){

    int unsortedArr[maxSize] = {5, 2, 9, 1, 5};
    srand(time(0));
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()% 100;
    }

    std::cout<<"Unsorted Array: \n"<<std::endl;
    displayArray(unsortedArr);

    std::cout<<"\nBubble Sort: \n"<<std::endl;
    bubbleSort(unsortedArr, maxSize);
    displayArray(unsortedArr);
}

void displayArray(int arr[]) {

    for(int i = 0; i < maxSize; i++){
        std::cout<<arr[i]<<" ";
    }
    std::cout<<std::endl;
}
```

```cpp
template<typename T>
void bubbleSort(T arr[], size_t arrSize) {
    for (int i = 0; i < arrSize; i++) {
        for (int j = i + 1; j < arrSize; j++) {
            if (arr[i] > arr[j]) {
                std::swap(arr[i], arr[j]);
            }
        }
    }
}
```

```
Bubble Sort:

0 1 2 3 5 6 6 8 9 10 10 11 11 12 14 14 15 15 15 16 16 17 17 19 21 22 23 24 27 29 29 30 30 31 32 33 34 34 36 37 37 39 39
40 41 41 42 42 43 44 48 48 49 49 49 50 52 53 54 54 56 57 60 61 61 61 62 63 63 64 65 68 68 69 70 71 71 71 72 73 74 75 75
77 77 79 80 81 81 83 87 88 89 89 92 92 93 95 96 98
PS C:\Users\pagan\OneDrive\Documents\Bungag 7p1>
```

Observations	The code uses Bubble Sort to arrange the 100 random numbers from smallest to largest. It compares the elements and swaps them whenever they are in the wrong order, resulting in a properly sorted array. The best-case and worst-case time complexity are both O(n²) because the nested loops still compare many elements even when the array is already sorted. With only 100 elements, the algorithm is still fast enough for this activity, but it becomes less efficient as the number of elements increases.

TABLE 7.3

Code + Console Screenshot	main.cpp > ...
	1 #include <iostream>
	2 #include <cstdlib>
	3 #include <time.h>
	4 #include "sortingAlgo.h"
	5
	6 const int maxSize = 100;
	7
	8 void displayArray(int arr[]);
	9
	10 int main(){
	11
	12 int unsortedArr[maxSize] = {5, 2, 9, 1, 5};
	13 srand(time(0));
	14 for(int i = 0; i < maxSize; i++){
	15 unsortedArr[i] = rand()% 100;
	16 }
	17
	18 std::cout<<"Unsorted Array: \n"<<std::endl;
	19 displayArray(unsortedArr);
	20
	21 //std::cout<<"\nBubble Sort: \n"<<std::endl;
	22 //bubbleSort(unsortedArr, maxSize);
	23 //displayArray(unsortedArr);
	24
	25 std::cout<<"\nSelection Sort: \n"<<std::endl;
	26 selectionSort(unsortedArr, maxSize);
	27 displayArray(unsortedArr);
	28 }
	29
	30 void displayArray(int arr[]) {
	31
	32 for(int i = 0; i < maxSize; i++){
	33 std::cout<<arr[i]<<" ";
	34 }
	35 std::cout<<std::endl;
	36 }
	37

```cpp
template <typename T>
int routineSmallest(T arr[], int K, const int arrSize) {
    int position = K;
    T smallestElem = arr[K];

    for (int j = K + 1; j < arrSize; j++) {
        if (arr[j] < smallestElem) {
            smallestElem = arr[j];
            position = j;
        }
    }
    return position;
}

template <typename T>
void selectionSort(T arr[], size_t arrSize) {
    int POS, temp, pass;
    for (int i = 0; i < arrSize; i++) {
        POS = routineSmallest(arr, i, arrSize);
        temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;
        pass++;
    }
}
```

```
Unsorted Array:

84 69 72 0 35 37 84 20 23 80 92 14 5 51 90 18 53 57 15 10 57 32 29 28 56 18 59 31 28
 30 25 55 11 36 99 78 76 58 58 57 40 5 23 12 79 48 10 21 32 59 77 10 16 6 32 13 47 3
4 12 68 91 0 53 41 65 14 93 39 81 11 34 97 54 11 29 62 78 5 67 54 3 39 12 69 26 7 63
 90 47 91 73 57 65 44 66 8 67 34 80 26

Selection Sort:

0 0 3 5 5 5 6 7 8 10 10 10 11 11 11 12 12 12 13 14 14 15 16 18 18 20 21 23 23 25 26
26 28 28 29 29 30 31 32 32 32 34 34 34 35 36 37 39 39 40 41 44 47 47 48 51 53 53 54
54 55 56 57 57 57 57 58 58 59 59 62 63 65 65 66 67 67 68 69 69 72 73 76 77 78 78 79
80 80 81 84 84 90 90 91 91 92 93 97 99
PS C:\Users\pagan\OneDrive\Documents\Bungag 7p1>
```

Observations	The code generates 100 random numbers and then uses Selection Sort to arrange them from smallest to largest. The routineSmallest() function searches for the smallest value in the unsorted part of the array and places it in the correct position. The best-case and worst-case time complexity are both $O(n^2)$ because Selection Sort still has to compare the remaining elements on every pass, even if the array is already sorted. For 100 elements, it works fine, but it can become slower when the array gets much larger.

TABLE 7.4

Code + Console Screenshot	main.cpp > ...
	1 #include <iostream>
	2 #include <cstdlib>
	3 #include <time.h>
	4 #include "sortingAlgo.h"
	5
	6 const int maxSize = 100;
	7
	8 void displayArray(int arr[]);
	9
	10 int main(){
	11
	12 int unsortedArr[maxSize] = {5, 2, 9, 1, 5};
	13 srand(time(0));
	14 for(int i = 0; i < maxSize; i++){
	15 unsortedArr[i] = rand()% 100;
	16 }
	17
	18 std::cout<<"Unsorted Array: \n"<<std::endl;
	19 displayArray(unsortedArr);
	20
	21 //std::cout<<"\nBubble Sort: \n"<<std::endl;
	22 //bubbleSort(unsortedArr, maxSize);
	23 //displayArray(unsortedArr);
	24
	25 //std::cout<<"\nSelection Sort: \n"<<std::endl;
	26 //selectionSort(unsortedArr, maxSize);
	27 //displayArray(unsortedArr);
	28
	29 std::cout<<"\nInsertion Sort: \n"<<std::endl;
	30 insertionSort(unsortedArr, maxSize);
	31 displayArray(unsortedArr);
	32 }
	33
	34 void displayArray(int arr[]) {
	35
	36 for(int i = 0; i < maxSize; i++){
	37 std::cout<<arr[i]<<" ";
	38 }
	39 std::cout<<std::endl;
	40 }

```
43  template <typename T>
44  void insertionSort(T arr[], size_t arrSize) {
45      int K = 0, J, temp;
46      while (K < arrSize) {
47          temp = arr[K];
48          J = K - 1;
49          while (temp <= arr[J]) {
50              arr[J + 1] = arr[J];
51              J--;
52          }
53          arr[J + 1] = temp;
54          K++;
55      }
56  }
57
58  #endif
```

```
Unsorted Array:

52 41 22 86 3 48 84 66 48 36 87 1 57 75 79 19 11 15 6 17 52 32 49 36 63 97 36 75 45 56 96 62 54 48
 15 49 89 95 42 53 47 11 81 49 56 84 80 67 76 25 81 88 74 84 26 5 91 78 5 80 29 75 25 38 60 25 86
30 33 24 60 54 49 53 36 57 80 43 48 57 90 35 92 41 78 37 80 69 1 52 79 40 30 26 9 55 92 96 26 1

Insertion Sort:

1 1 1 3 5 5 6 9 11 11 15 15 17 19 22 24 25 25 25 26 26 26 29 30 30 32 33 35 36 36 36 36 37 38 40 4
1 41 42 43 45 47 48 48 48 48 49 49 49 49 52 52 52 53 53 54 54 55 56 56 57 57 57 60 60 62 63 66 67
69 74 75 75 75 76 78 78 79 79 80 80 80 80 81 81 84 84 84 86 86 87 88 89 90 91 92 92 95 96 96 97
PS C:\Users\pagan\OneDrive\Documents\Bungag 7p1>
```

Observations	The code generates 100 random numbers and uses Insertion Sort to arrange them from smallest to largest. It works by taking each element and inserting it into its correct position among the elements that have already been sorted. The best-case time complexity is O(n) when the array is already sorted, because only a few comparisons are needed. The worst-case time complexity is O(n²) when the array is in reverse order, because many elements need to be compared and shifted. For 100 elements, the algorithm works well, but it becomes slower as the array size increases.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Pseudocode

```
START

SET SIZE = 100

CREATE an array called votes with 100 elements

GENERATE 100 random votes
    Each vote is a number from 1 to 5

DISPLAY the unsorted array

SELECTION SORT
    FOR i from 0 to SIZE - 1
        SET smallest = i

        FOR j from i + 1 to SIZE - 1
            If votes[j] is smaller than votes[smallest]
                SET smallest = j
            END If
        END For

        SWAP votes[i] and votes[smallest]
    END For

DISPLAY the sorted array

COUNT THE VOTES
    SET all candidate counts to 0

    For each vote in the array
        Add 1 to the count of that candidate
    END For

DISPLAY the vote count of Candidates 1 to 5

FIND THE WINNER
    SET Candidate 1 as the current winner

    COMPARE the vote counts of Candidates 2 to 5
    If a candidate has more votes than the current winner
        SET that candidate as the winner
    END If

DISPLAY the winning candidate and their votes

END
```

Code

selectionsortsupp.h > routineSmallest<T>(T [], int, const int)

```cpp
#include <cstddef>
#include <iostream>

#ifndef SELECTIONSORTSUPP_H
#define SELECTIONSORTSUPP_H

template <typename T>
int routineSmallest(T arr[], int K, const int arrSize) {
    int position = K;
    T smallestElem = arr[K];

    for (int j = K + 1; j < arrSize; j++) {
        if (arr[j] < smallestElem) {
            smallestElem = arr[j];
            position = j;
        }
    }
    return position;
}

template <typename T>
void selectionSort(T arr[], size_t arrSize) {
    int POS, temp, pass;
    for (int i = 0; i < arrSize; i++) {
        POS = routineSmallest(arr, i, arrSize);
        temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;
        pass++;
    }
}
#endif
```

suppmain.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "selectionsortsupp.h"

const int maxSize = 101;
void displayArray(int arr[]) {
    for(int i = 0; i < maxSize; i++){
        std::cout<<arr[i]<<" ";
    }
    std::cout<<std::endl;
}

int main(){

    int votes[maxSize];
    srand(time(0));
    for (int i = 0; i < maxSize; i++) {
        votes[i] = rand() % 5 + 1;
    }

    std::cout << "Unsorted Array:\n";
    displayArray(votes);

    selectionSort(votes, maxSize);

    std::cout <<"\nSorted Array:\n";
    displayArray(votes);

    int count1 = 0;
    int count2 = 0;
    int count3 = 0;
    int count4 = 0;
    int count5 = 0;

    for (int i = 0; i < maxSize; i++) {

        if (votes[i] == 1)
            count1++;
        else if (votes[i] == 2)
            count2++;
        else if (votes[i] == 3)
            count3++;
        else if (votes[i] == 4)
            count4++;
        else if (votes[i] == 5)
            count5++;
    }
```

```
    std::cout <<"\nVote Count:\n";
    std::cout <<"Candidate 1: Bo Dalton Capistrano: "<<count1 <<"\n";
    std::cout <<"Candidate 2: Cornelius Raymon Agustin: "<<count2 <<"\n";
    std::cout <<"Candidate 3: Deja Jayla Banaga: "<< count3 <<"\n";
    std::cout <<"Candidate 4: Lalla Brielle Yabut: "<< count4 <<"\n";
    std::cout <<"Candidate 5: Franklin Relano Castro: "<< count5 <<"\n";

    int highest = count1;
    int winner = 1;

    if (count2 > highest) {
        highest = count2;
        winner = 2;
    }
    if (count3 > highest) {
        highest = count3;
        winner = 3;
    }
    if (count4 > highest) {
        highest = count4;
        winner = 4;
    }
    if (count5 > highest) {
        highest = count5;
        winner = 5;
    }

    std::cout <<"\nThe Winner is Candidate "<< winner <<" with "<< highest <<" votes!\n";
    return 0;
}
```

Console

```
Unsorted Array:
2 1 3 3 2 2 3 1 4 4 2 1 3 1 1 5 1 2 3 1 2 1 5 2 3 2 3 4 3 4 4 4 2 2 2 2 5 2 4 3 4 5 1 4 3 4 2 1 1 2 4 4 2 1 4 4 2 4
 2 2 3 2 5 1 4 4 5 4 5 4 2 1 1 4 2 4 5 1 5 5 3 4 1 3 3 2 5 3 1 1 5 4 5 3 4 1 4 1 2 1 1

Sorted Array:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3
 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5

Vote Count:
Candidate 1: Bo Dalton Capistrano: 23
Candidate 2: Cornelius Raymon Agustin: 24
Candidate 3: Deja Jayla Banaga: 16
Candidate 4: Lalla Brielle Yabut: 25
Candidate 5: Franklin Relano Castro: 13

The Winner is Candidate 4 with 25 votes!
```

QUESTION:

The voting system is effective, the vote counting algorithm was effective because it correctly sorted the 100 votes using selection sort and counted how many votes each candidate received. It also made it easy to identify the candidate with the highest number of votes. The algorithm worked as expected and gave accurate results.

CONCLUSION:

Overall, this activity helped me understand how different sorting algorithms work and how they affect the performance of a program. I learned how Bubble Sort, Selection Sort, and Insertion Sort arrange data in different ways, even though they can produce the same sorted result. I also learned that their time complexities are different depending on the situation. Bubble Sort and Selection Sort have $O(n^2)$ time complexity in both the best and worst cases, while Insertion Sort can be faster with a best case of $O(n)$ but still has $O(n^2)$ in the worst case. Through implementing and testing these algorithms, I understood that choosing the right sorting method depends on the amount and arrangement of data. This activity also improved my understanding of arrays, loops, comparisons, and how algorithms can affect the efficiency of a program.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.